C++ PREVIOUS YEAR QUESTIONS AND ANSWERS

Q1). What are virtual functions and what are its uses ? Explain with example. What are Pure virtual functions and what are its uses ? Explain with example.

1. Virtual Functions

Definition:
A virtual function is a member function in a base class declared with the keyword virtual. It allows runtime polymorphism, meaning that the function to be called is determined at runtime based on the actual object type, even if accessed through a base class pointer or reference.

Uses:

- Enables dynamic binding instead of static (compile-time) binding.
- Supports polymorphism, allowing the correct overridden function in derived classes to be called via base class pointers.
- Useful in cases where multiple derived classes provide different implementations of the same function.
- Helps design flexible and maintainable programs.

Example:

cpp
Copy code

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() {  // Virtual function
        cout << "Animal makes a sound" << endl;
    }
};

class Dog : public Animal {
public:
    void sound() override {  // Overrides base class function
        cout << "Dog barks" << endl;
    }
};

int main() {
    Animal* animal = new Dog();
    animal->sound();  // Calls Dog's sound() due to virtual function
    delete animal;
    return 0;
}
```

Output:
Dog barks

---

2. Pure Virtual Functions

Definition:
A pure virtual function is a virtual function that has no implementation in the base class and is declared by assigning = 0 in its declaration. It makes the base class an abstract class which cannot be instantiated.

Uses:

- Used to create abstract base classes.
- Forces derived classes to provide their own implementation of the function.
- Helps define interfaces or common behavior without providing default implementation.
- Enables designing polymorphic class hierarchies where base class acts as a template.

Example:

cpp
Copy code

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() = 0;  // Pure virtual function
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle" << endl;
    }
};

int main() {
    // Shape s; // Error: Cannot instantiate abstract class
    Shape* shape = new Circle();
    shape->draw();  // Calls Circle's draw()
    delete shape;
    return 0;
}
```

Output:
Drawing Circle

Summary Table

Concept	Definition	Purpose/Use	Example Function Declaration
Virtual Function	Virtual function with implementation	Supports runtime polymorphism and dynamic binding	virtual void sound();
Pure Virtual Function	Virtual function with no implementation (=0)	Defines abstract class, forces override in derived classes	virtual void draw() = 0;

Conclusion

- Virtual functions provide dynamic dispatch and allow derived class methods to be called through base class pointers.

- Pure virtual functions create abstract classes and enforce implementation of functions in derived classes.
- Both concepts are fundamental to polymorphism in C++ and help design flexible and reusable code.

Q2). What is a friend function ? How does it differ from the member function ? Explain the implementation of friend function in C++, with the help of an example program.

What is a Friend Function?

A friend function is a special function that is not a member of a class but is allowed to access the private and protected members of that class. It is declared inside a class with the keyword friend.

---

2. Difference Between Friend Function and Member Function

Aspect	Friend Function	Member Function
Membership	Not a member of the class	Is a member of the class
Access	Can access private and protected members when declared as friend	Can access all members (private, protected, public)
Invocation	Called like a normal function, not using object	Called using object or pointer with dot or arrow operator
Declared using	friend keyword inside class declaration	No special keyword needed
Scope	Has no this pointer (since not a member)	Has this pointer referring to calling object

---

3. Uses of Friend Functions

- To access private or protected data members of a class from a non-member function.
- Useful when two or more classes need to access each other's private members (can be friend of multiple classes).
- Helps in operator overloading for operators where the left operand is not an object of the class.

---

4. Implementation of Friend Function in C++ (Example)

cpp
Copy code

```
#include <iostream>
using namespace std;

class Box {
private:
   int length;

public:
   Box() {
      length = 0;
   }

   // Friend function declaration
   friend void setLength(Box& b, int len);

   void display() {
```

```
        cout << "Length of box: " << length << endl;
    }
};

// Friend function definition
void setLength(Box& b, int len) {
    b.length = len;  // Accessing private member length directly
}

int main() {
    Box box1;

    setLength(box1, 10);  // Using friend function to set private data

    box1.display();

    return 0;
}
```

Output:

rust
Copy code
Length of box: 10

---

5. Explanation:

- The function setLength() is declared as a friend of class Box using friend keyword.
- Although setLength() is not a member function, it can access the private data member length directly.
- In main(), the friend function is called normally without using any object or pointer.
- The friend function modifies private data, showing how friend functions can access class internals.

3. Differentiate between member functions and global functions with help of an example.

Difference Between Member Functions and Global Functions

Aspect	Member Functions	Global Functions
Definition	Functions that are defined inside a class and operate on class objects	Functions defined outside any class, not associated with any class
Access	Can access all members (private, protected, public) of the class directly	Cannot access private/protected members of a class directly
Invocation	Called using an object or pointer with dot (.) or arrow (->) operator	Called directly by their name or with scope if needed
Scope	Part of the class scope and can use the this pointer to access object members	Part of global scope (or namespace) and has no this pointer
Use	Used to perform operations on class data or provide behavior for class objects	Used for general utility or functions that do not belong to any specific class
Access to this	Has this pointer referring to the invoking object	Does not have this pointer

---

Example

cpp
Copy code

```cpp
#include <iostream>
using namespace std;

class Box {
private:
    int length;

public:
    Box(int l) : length(l) {}

    // Member function: can access private member directly
    void displayLength() {
        cout << "Length is: " << length << endl;
    }

    // Declare friend function (a kind of global function with special access)
    friend void globalDisplayLength(Box& b);
};

// Global function: not part of class, no direct access to private members unless friend
void globalDisplayLength(Box& b) {
    cout << "Length accessed by global function: " << b.length << endl;
}

int main() {
    Box box1(10);

    box1.displayLength();      // Member function called using object
    globalDisplayLength(box1); // Global function called normally, accesses private data via friendship

    return 0;
}
```

Output:

vbnet
Copy code

```
Length is: 10
Length accessed by global function: 10
```

Explanation:

- displayLength() is a member function and can access length directly via this pointer.
- globalDisplayLength() is a global function but declared as a friend so it can access private members.
- Normally, global functions cannot access private members unless declared friend

Q4). What is an inline function ? How does the execution of inline functions differ from normal functions ? Explain the advantages of inline function, with suitable example

. What is an Inline Function?

An inline function is a function where the compiler expands the function code at the point of call instead of performing a normal function call. It is suggested to the compiler by using the keyword inline before the function definition.

---

2. Execution Difference Between Inline and Normal Functions

Aspect	Inline Function	Normal Function
Function Call	The function body is inserted directly at the call site (code expansion)	A function call is made (stack overhead occurs)
Overhead	No function call overhead (no pushing/popping stack)	Has overhead of function call (stack operations)
Execution Speed	Faster execution due to no call overhead	Slower due to call and return instructions
Code Size	May increase code size due to multiple copies of function body	Smaller code size, only one copy of function code

---

3. Advantages of Inline Functions

- Faster execution by avoiding function call overhead.
- Useful for small and frequently called functions.
- Helps reduce function call overhead in performance-critical sections.
- Encourages better code readability by allowing function use without performance penalty.

---

4. Example of Inline Function

cpp
Copy code

```
#include <iostream>
using namespace std;

inline int square(int x) {  // Inline function
    return x * x;
}

int main() {
    int num = 5;
    cout << "Square of " << num << " is " << square(num) << endl;
    return 0;
}
```

Output:

csharp
Copy code

```
Square of 5 is 25
```

---

5. Explanation:

- The function square() is declared as inline.
- Instead of calling the function normally, the compiler replaces the call square(num) with num * num.

- This reduces the overhead of function calls and improves execution speed, especially for small functions like this.

Q5). What is object oriented programming ? Explain the benefits of object oriented programming. How does object oriented programming differ from structured programming ?

Object Oriented Programming (OOP) (15 Marks)

---

1. What is Object Oriented Programming?

Object Oriented Programming (OOP) is a programming paradigm based on the concept of objects. Objects are instances of classes that combine data (attributes) and functions (methods) to represent real-world entities. OOP focuses on organizing code into reusable and interacting objects.

Key features of OOP are:

- Encapsulation: Bundling data and methods that operate on data within one unit (class).
- Inheritance: Deriving new classes from existing ones, inheriting properties and behavior.
- Polymorphism: Ability to process objects differently based on their data type or class.
- Abstraction: Hiding complex implementation details and showing only the necessary features.

---

2. Benefits of Object Oriented Programming

- Modularity: Code is organized into classes/objects, making it easier to manage and maintain.
- Reusability: Classes can be reused through inheritance and composition, reducing redundancy.
- Flexibility and Scalability: Polymorphism and inheritance allow flexible code extension without modifying existing code.
- Maintainability: Easier to update and debug due to clear structure and encapsulation.
- Real-world Modeling: Represents real-world entities more naturally using objects.
- Data Security: Encapsulation protects data by restricting direct access through access specifiers (private, protected).

---

3. Difference Between Object Oriented Programming and Structured Programming

Aspect	Object Oriented Programming (OOP)	Structured Programming
Basic Unit	Object (combines data and functions)	Function or procedure
Focus	Data and functions together (encapsulation)	Sequence of instructions or functions
Data Handling	Data is hidden and accessed through methods	Data and functions are separate
Code Reusability	Achieved through inheritance and polymorphism	Limited reusability through functions
Approach	Bottom-up approach	Top-down approach
Real-world Modeling	Better modeling using objects and classes	Less suitable for complex real-world problems

Aspect	Object Oriented Programming (OOP)	Structured Programming
Data Security	Provides data hiding (encapsulation)	No direct support for data hiding
Example Languages	C++, Java, Python (OOP-based languages)	C, Pascal, BASIC (procedural languages)

Q6). Define the term class. How are class and object related ? Explain with example.

. Definition of Class

A class is a user-defined data type in C++ that serves as a blueprint or template for creating objects. It encapsulates data members (variables) and member functions (methods) that operate on the data. A class defines the properties and behaviors that its objects will have.

---

2. Relationship Between Class and Object

- A class is a blueprint or template.
- An object is an instance of a class.
- Objects are created based on the structure defined by the class.
- Multiple objects can be created from a single class, each with its own copy of data members.

---

3. Example

cpp
Copy code

```
#include <iostream>
using namespace std;

// Class definition
class Car {
public:
    string brand;
    int year;

    void display() {
        cout << "Brand: " << brand << ", Year: " << year << endl;
    }
};

int main() {
    // Creating objects of class Car
    Car car1;
    Car car2;

    // Assign values to car1
    car1.brand = "Toyota";
    car1.year = 2015;

    // Assign values to car2
    car2.brand = "Honda";
    car2.year = 2018;

    // Display details
```

```
    car1.display();
    car2.display();

    return 0;
}
```

7. Compare Class templates and Function templates with the help of example code.

Comparison Between Class Templates and Function Templates

Aspect	Class Template	Function Template
Definition	A blueprint to create a class for any data type	A blueprint to create a function for any data type
Purpose	To create generic classes that work with any data type	To create generic functions that work with any data type
Syntax	template <typename T> class ClassName { ... };	template <typename T> ReturnType functionName(...) { ... }
Usage	Used when you want a class with data members or methods generalized	Used when you want a function that performs the same operation for multiple data types
Example Scenario	Generic container classes like Array, Stack, LinkedList	Generic functions like swap(), max(), min()
Instantiation	Objects of the template class are created with specific types	Functions are called with specific types automatically deduced

---

Example of Class Template

cpp
Copy code

```
#include <iostream>
using namespace std;

template <typename T>
class Box {
private:
    T length;

public:
    Box(T l) : length(l) {}

    void display() {
        cout << "Length: " << length << endl;
    }
};

int main() {
    Box<int> intBox(10);       // Class template instantiated with int
    Box<double> doubleBox(5.5); // Class template instantiated with double

    intBox.display();
```

```
doubleBox.display();

    return 0;
}
```

Output:

makefile
Copy code
Length: 10
Length: 5.5

Example of Function Template

cpp
Copy code

```
#include <iostream>
using namespace std;

template <typename T>
T getMax(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << getMax<int>(3, 7) << endl;      // Function template with int
    cout << getMax<double>(3.5, 2.1) << endl; // Function template with double
    cout << getMax<char>('a', 'z') << endl;   // Function template with char

    return 0;
}
```

Q8). What are constructors ? Write the characteristics of a constructor. What are the limitations of a constructor ? What is a Destructor in C++ ? Do constructors and destructors have return type ?

What is a Constructor?

A constructor is a special member function of a class that is automatically called when an object is created. Its main purpose is to initialize the object (i.e., assign default or user-defined values to data members).

 Characteristics of Constructors

1. Same name as the class
   A constructor must have the same name as the class it belongs to.
2. No return type
   Constructors do not have any return type, not even void.
3. Automatically invoked
   It is called automatically when an object is created.
4. Can be overloaded
   You can have multiple constructors in the same class with different parameters (constructor overloading).
5. Can have default arguments
   Constructors can have default parameters.
6. Cannot be inherited
   Constructors are not inherited by derived classes.

7. Can be parameterized
   Constructors can accept arguments to initialize an object with specific values.

 Example of Constructor

cpp
Copy code

```cpp
#include <iostream>
using namespace std;

class Student {
    int roll;
    string name;

public:
    // Constructor
    Student(int r, string n) {
        roll = r;
        name = n;
    }

    void display() {
        cout << "Roll: " << roll << ", Name: " << name << endl;
    }
};

int main() {
    Student s1(101, "Amit");  // Constructor is called here
    s1.display();

    return 0;
}
```

Output:

yaml
Copy code

```yaml
Roll: 101, Name: Amit
```

 Limitations of Constructor

1. Cannot be virtual – Constructors cannot be declared as virtual.
2. No return value – Constructors cannot return values or be used in expressions.
3. No constructor overloading with default values and similar signatures – May create ambiguity.
4. Can't be inherited – Constructors from base class are not inherited by derived class directly.
5. Limited in multiple inheritance – When multiple base classes have constructors, the order of constructor calls must be carefully managed.

 What is a Destructor?

A destructor is a special member function that is automatically called when an object is destroyed. It is used to free resources, such as memory or file handles, that were allocated to the object during its lifetime.

Syntax:

cpp
Copy code
```
~ClassName() {
    // cleanup code
}
```

Q9).Differentiate between copy constructor and default constructor in C++, with the help of an example for each

What is a Default Constructor?

A default constructor is a constructor that takes no parameters (or all parameters have default values). It is used to create and initialize objects with default values.

 What is a Copy Constructor?

A copy constructor is a special constructor that creates a new object by copying values from an existing object. It takes a reference to an object of the same class as a parameter.

 Difference Between Default Constructor and Copy Constructor

Feature	Default Constructor	Copy Constructor
Definition	Constructor with no parameters	Constructor with one parameter — reference of same class
Purpose	Initializes object with default or fixed values	Creates a new object as a copy of an existing object
Invocation	Called when object is created without arguments	Called when object is initialized with another object
Provided by Compiler	Automatically provided if no constructor is defined	Automatically provided unless user-defined
Parameters	None (or all defaulted)	One parameter of same class by reference
Overloading	Can be overloaded	Cannot be overloaded

 Example of Default Constructor

cpp
Copy code
```
#include <iostream>
using namespace std;

class Student {
    int roll;
public:
    Student() {         // Default constructor
        roll = 0;
    }

    void display() {
        cout << "Roll: " << roll << endl;
    }
```

```
};

int main() {
    Student s1;         // Calls default constructor
    s1.display();
    return 0;
}
```

Output:

makefile
Copy code
Roll: 0

 Example of Copy Constructor

cpp
Copy code

```
#include <iostream>
using namespace std;

class Student {
    int roll;
public:
    Student(int r) {
        roll = r;
    }

    Student(const Student &s) {  // Copy constructor
        roll = s.roll;
    }

    void display() {
        cout << "Roll: " << roll << endl;
    }
};

int main() {
    Student s1(101);     // Normal constructor
    Student s2 = s1;     // Copy constructor called

    s1.display();
    s2.display();
    return 0;
}
```

Q10). Short note on: Default and Parameterized Constructor. New and Delete Operator

Default Constructor

A default constructor is a constructor that takes no parameters. It is automatically called when an object is created without any arguments. If no constructor is defined by the programmer, the compiler provides a default constructor.

Example:

cpp
Copy code

```
class Demo {
public:
    Demo() {
        cout << "Default Constructor Called" << endl;
    }
};
```

Use: To initialize objects with default values.

---

2. □ Parameterized Constructor

A parameterized constructor is a constructor that takes arguments. It is used to initialize an object with specific values at the time of its creation.

Example:

```
cpp
Copy code
class Demo {
    int x;
public:
    Demo(int a) {
        x = a;
    }
};
```

Use: Allows assigning values to data members during object creation: Demo obj(10);

---

3. □ new and delete Operators

These are dynamic memory management operators in C++.

new Operator:

Allocates memory at runtime and returns a pointer to the allocated memory.

Syntax:

```
cpp
Copy code
int* ptr = new int;       // allocates memory for an integer
```

Use: Similar to malloc() in C, but safer and type-aware.

delete Operator:

Releases the memory allocated using new.

Syntax:

cpp
Copy code
delete ptr;

Example:

cpp
Copy code
int* p = new int(10);
cout << *p << endl;
delete p;

Q11). What is STL ? Explain the importance of STL library in C++.

What is STL in C++?

STL stands for Standard Template Library.
It is a powerful library in C++ that provides a set of generic classes and functions for handling data structures and algorithms.

STL makes use of templates to provide reusable and type-independent code.

Components of STL

STL has 3 main components:

Component	Description
Containers	Used to store collections of objects (e.g., vector, list, map)
Algorithms	Built-in functions to process data in containers (e.g., sort(), find())
Iterators	Used to access elements in containers, similar to pointers

---

Importance of STL in C++

1. Reusability of Code
   You don't need to write your own data structures or sorting logic — STL provides ready-made tools.
2. Faster Development
   Speeds up program development since common operations are already implemented.
3. Efficiency
   STL components are well-tested, optimized, and efficient.
4. Type Safety with Templates
   STL uses templates, which means you can use the same container for any data type.
5. Consistency
   STL provides a uniform way to interact with different data structures using iterators.
6. Portability
   STL is part of the C++ Standard Library, so STL-based code works across platforms.

✅ Example: Using vector and sort() from STL

cpp
Copy code

```
#include <iostream>
#include <vector>
#include <algorithm> // for sort()

using namespace std;

int main() {
    vector<int> nums = {5, 2, 8, 1};

    sort(nums.begin(), nums.end());  // STL algorithm

    for(int n : nums)
        cout << n << " ";

    return 0;
}
```

Q12). What is Type Conversion ? What is the advantage of Type Conversion ? Briefly discuss Type Casting and Automatic Type Conversion.

What is Type Conversion?

Type Conversion is the process of converting one data type into another. In C++, it allows a variable of one data type to be used as a different data type, either automatically or manually.

✅ Advantage of Type Conversion

- Ensures compatibility during arithmetic operations.
- Helps in reducing data loss during conversion.
- Improves program flexibility and correctness.
- Enables safe mixing of different data types in expressions.

Types of Type Conversion in C++

C++ supports two main types:

1. Implicit Type Conversion (Automatic Type Conversion)

- Done automatically by the compiler.
- Happens when you assign a smaller data type to a larger data type.

Example:

cpp
Copy code

```
int a = 10;
float b = a;  // int is automatically converted to float
```

Key Point: No data loss if converting from smaller to larger type (e.g., int to float).

---

2. Explicit Type Conversion (Type Casting)

- Done manually by the programmer using casting operators.
- Used when automatic conversion is not suitable or safe.

Syntax:
cpp
Copy code
dataType(variable)

Example:
cpp
Copy code
float a = 5.75;
int b = (int)a; // explicit type casting

Q13). What is Polymorphism ? What are the advantages of polymorphism ? Mention the types of polymorphism supported by C++.

? What is Polymorphism?

Polymorphism is a key feature of Object-Oriented Programming (OOP) that means "many forms". In C++, polymorphism allows the same function name or operator to behave differently based on the context (input types or objects involved).

It enables one interface to be used for different underlying forms (data types or functions).

? Advantages of Polymorphism

1. □ Code Reusability – You can write generic code that works for multiple data types or objects.
2. □ Flexibility & Maintainability – Allows extending programs without rewriting existing code.
3. □ Simplified Code – Common interfaces reduce complexity and improve clarity.
4. □ Runtime Decision Making – Useful for real-time decision-making via virtual functions.
5. □ Supports Extensibility – New functionality can be added with minimal changes.

? Types of Polymorphism in C++

C++ supports two main types of polymorphism:

Type	Description	When Decided
Compile-time Polymorphism	Function/operator overloading (early binding)	At compile time
Runtime Polymorphism	Function overriding using virtual functions	At runtime

---

 1. Compile-Time Polymorphism (Static Polymorphism)

This occurs when function calls are resolved during compilation. It is achieved using:

a) Function Overloading

Same function name with different parameter lists.

cpp
Copy code
```
void display(int a) { cout << "Integer: " << a; }
void display(float b) { cout << "Float: " << b; }
```

b) Operator Overloading

Custom behavior for standard operators.

cpp
Copy code
```
class Complex {
    int x;
public:
    Complex(int val) { x = val; }
    Complex operator+(const Complex &obj) {
        return Complex(x + obj.x);
    }
};
```

---

 2. Runtime Polymorphism (Dynamic Polymorphism)

Occurs when function call is resolved at runtime using virtual functions and inheritance.

Example:

cpp
Copy code
```
class Base {
public:
    virtual void show() {
        cout << "Base class" << endl;
    }
};

class Derived : public Base {
public:
    void show() override {
        cout << "Derived class" << endl;
    }
};

int main() {
    Base* ptr;
    Derived d;
    ptr = &d;
    ptr->show();  // Calls Derived::show() (runtime polymorphism)
}
```

Q14). What is Encapsulation ? Explain the difference between information hiding and encapsulation with the help of an example.

What is Encapsulation?

Encapsulation is one of the fundamental concepts of Object-Oriented Programming (OOP). It refers to the bundling of data and the functions that operate on that data into a single unit (class), and restricting direct access to some components of the object.

In simple terms: Encapsulation = Data + Code + Access Control

---

Benefits of Encapsulation:

1. Protects data from unauthorized access.
2. Helps in modular, maintainable code.
3. Improves security and flexibility.
4. Simplifies testing and debugging.

What is Information Hiding?

Information hiding means restricting access to internal details of a class or object — such as data members — to protect its integrity.

It's achieved in C++ using access specifiers like private, protected, and public.

Difference Between Encapsulation and Information Hiding

Feature	Encapsulation	Information Hiding
Definition	Wrapping data and functions in a class	Hiding internal details from outside access
Purpose	To group code and data together	To prevent misuse of data
Means	Achieved through classes & access modifiers	Achieved through private/protected access
Visibility	Controls "what to show" and "how to access"	Controls "what not to show"
Example Focus	Structure of a class	Restriction of internal data

---

Example in C++: Encapsulation & Information Hiding

cpp
Copy code

```
#include <iostream>
using namespace std;

class BankAccount {
private:
```

```
    double balance;  // Hidden from outside (Information hiding)

public:
    // Constructor
    BankAccount(double initialBalance) {
        balance = initialBalance;
    }

    // Public method to access private data (Encapsulation)
    void deposit(double amount) {
        if (amount > 0)
            balance += amount;
    }

    void displayBalance() {
        cout << "Current Balance: " << balance << endl;
    }
};

int main() {
    BankAccount account(1000);

    // account.balance = 5000;  //  Not allowed (private)
    account.deposit(500);       //  Allowed via public method
    account.displayBalance();

    return 0;
}
```

Q15). What is inheritance ? Briefly discuss the different types of inheritance, supported by C++. Explain how inheritance is implemented in C++.

What is Inheritance in C++?

Inheritance is a key concept of Object-Oriented Programming (OOP) that allows a class (called derived class) to acquire the properties and behaviors (data and functions) of another class (called base class).

 It supports code reusability, extensibility, and hierarchical classification.

Syntax of Inheritance in C++:

cpp
Copy code

```
class Base {
    // members
};

class Derived : accessSpecifier Base {
    // additional members
};
```

accessSpecifier can be public, protected, or private.

Example:

cpp

Copy code
```
#include <iostream>
using namespace std;

class Animal {
public:
    void eat() {
        cout << "Eating..." << endl;
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Barking..." << endl;
    }
};

int main() {
    Dog d;
    d.eat();   // Inherited from Animal
    d.bark();  // Defined in Dog
    return 0;
}
```

 Types of Inheritance in C++

Type	Description
Single Inheritance	One derived class inherits from one base class
Multilevel Inheritance	A derived class inherits from another derived class
Multiple Inheritance	One derived class inherits from more than one base class
Hierarchical Inheritance	Multiple derived classes inherit from a single base class
Hybrid Inheritance	A combination of two or more types of inheritance

---

 1. Single Inheritance

cpp
Copy code
```
class A {
public: void show() { cout << "A"; }
};
class B : public A {
public: void display() { cout << "B"; }
};
```

---

 2. Multilevel Inheritance

cpp
Copy code
```
class A { };
```

```
class B : public A { };
class C : public B { };
```

---

 3. Multiple Inheritance

```
cpp
Copy code
class A { };
class B { };
class C : public A, public B { };
```

---

 4. Hierarchical Inheritance

```
cpp
Copy code
class A { };
class B : public A { };
class C : public A { };
```

---

 5. Hybrid Inheritance

A combination of any of the above forms.

```
cpp
Copy code
class A { };
class B : public A { };
class C { };
class D : public B, public C { };
```

 How Inheritance is Implemented in C++

- Inheritance is implemented using the : symbol in class declaration.
- Members of the base class are accessed in the derived class depending on the access specifier used:

Access Specifier	Public Members	Protected Members	Private Members
public	Inherited as public	Inherited as protected	Not inherited
protected	Inherited as protected	Inherited as protected	Not inherited
private	Not accessible	Not accessible	Not accessible

Q16). What is an access specifier ? Explain different types of access specifiers available in C++.

What is an Access Specifier in C++?

Access Specifiers in C++ define the visibility and accessibility of class members (variables and functions) from outside the class.
They determine which parts of the program can access or modify the members of a class.

 C++ provides three main access specifiers: private, protected, and public.

 Types of Access Specifiers in C++

Access Specifier	Accessible Within Class	Accessible in Derived Class	Accessible Outside Class
private	Yes	No	No
protected	Yes	Yes	No
public	Yes	Yes	Yes

---

 1. private

- Members declared as private can only be accessed inside the class.
- Not accessible by derived classes or outside the class.

Example:

cpp
Copy code
```
class Student {
private:
   int marks;
public:
   void setMarks(int m) { marks = m; }
   void display() { cout << "Marks: " << marks; }
};
```

---

 2. protected

- Members declared as protected are accessible inside the class and by derived classes, but not outside the class.

Example:

cpp
Copy code
```
class Base {
protected:
   int x;
};

class Derived : public Base {
public:
   void setX(int a) { x = a; }  // allowed
};
```

---

 3. public

- Members declared as public can be accessed from anywhere: inside the class, derived class, and from outside.

Example:

cpp
Copy code
```
class Book {
public:
   string title;
   void display() {
      cout << "Title: " << title << endl;
   }
};
```

□ Why Use Access Specifiers?

1. □ Encapsulation: Protects internal data.
2. □□ Data Hiding: Prevents misuse by unauthorized access.
3. □ Inheritance Control: Defines how members behave in derived classes.
4. □ Code Maintainability: Easier to manage with clear access boundaries.

□ Example Combining All Specifiers

cpp
Copy code
```
class Example {
private:
   int a;  // not accessible outside
protected:
   int b;  // accessible in derived class
public:
   int c;  // accessible everywhere

   void setValues() {
      a = 10;
      b = 20;
      c = 30;
   }
};
```

Q17). Differentiate between early binding and late binding with an example of each

What is Binding in C++?

Binding refers to the process of connecting a function call to the function definition. In C++, this can happen at compile time (early binding) or at runtime (late binding).

□ Difference Between Early Binding and Late Binding

Feature	Early Binding (Static Binding)	Late Binding (Dynamic Binding)
Time of Binding	Happens at compile time	Happens at runtime

Feature	Early Binding (Static Binding)	Late Binding (Dynamic Binding)
Performance	Faster (no runtime overhead)	Slightly slower (uses virtual table)
Flexibility	Less flexible	More flexible
Used With	Normal functions, overloaded functions	Virtual functions, overridden functions
Polymorphism Type	Compile-time polymorphism	Runtime polymorphism
Keyword Required	No keyword required	Uses virtual keyword

 Example of Early Binding

cpp
Copy code

```
#include <iostream>
using namespace std;

class Math {
public:
    void add(int a, int b) {
        cout << "Sum: " << a + b << endl;
    }
};

int main() {
    Math m;
    m.add(5, 3);  // Early binding — resolved at compile time
    return 0;
}
```

 Explanation:

- The compiler knows at compile time which add() function to call.
- No use of inheritance or virtual functions.

 Example of Late Binding

cpp
Copy code

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() {
        cout << "Animal sound" << endl;
    }
};

class Dog : public Animal {
public:
    void sound() override {
        cout << "Dog barks" << endl;
    }
```

```
};

int main() {
    Animal* a;
    Dog d;
    a = &d;
    a->sound();  // Late binding — resolved at runtime using virtual function
    return 0;
}
```

 Explanation:

- sound() is marked as virtual.
- Compiler defers the decision until runtime based on the actual object (Dog), not the pointer type (Animal).

 Conclusion

- Early Binding improves speed but is less flexible.
- Late Binding provides flexibility and supports runtime polymorphism, but has slight overhead.
- Use virtual functions for late binding when dynamic behavior is needed.

.Q18). What are Stream Manipulators ? Briefly discuss the purpose of various stream manipulators.

What are Stream Manipulators?

Stream manipulators in C++ are special functions or objects used to modify the behavior of input/output streams (cin, cout, etc.) in a convenient way. They help format the data being displayed or read without needing complex formatting code.

Manipulators are used with the insertion (<<) or extraction (>>) operators to control how data is presented or interpreted, such as setting the number of decimal places, adjusting field width, or changing the number base.

Purpose of Various Stream Manipulators

Here are some commonly used stream manipulators and their purposes:

Manipulator	Purpose	Example Usage
endl	Inserts a newline character and flushes the output buffer	cout << "Hello" << endl;
setw(int n)	Sets the field width to n for the next output	cout << setw(10) << x;
setprecision(int n)	Sets the number of digits to display for floating-point values	cout << setprecision(3) << pi;
fixed	Displays floating-point numbers in fixed-point notation	cout << fixed << 12.3456;
scientific	Displays floating-point numbers in scientific notation	cout << scientific << 12.3456;
left	Left-aligns the output within the specified width	cout << left << setw(10) << x;
right	Right-aligns the output within the specified width	cout << right << setw(10) << x;
hex	Displays integers in hexadecimal format	cout << hex << 255;
dec	Displays integers in decimal format	cout << dec << 255;
oct	Displays integers in octal format	cout << oct << 255;
showpoint	Forces the output to show the decimal point for floats	cout << showpoint << 3.0;
noshowpoint	Disables the decimal point display if not necessary	cout << noshowpoint;

Manipulator	Purpose	Example Usage
boolalpha	Displays boolean values as true or false instead of 1 or 0	cout << boolalpha << true;
noboolalpha	Displays boolean values as 1 or 0	cout << noboolalpha << true;

Q19).What is message passing ? Explain with the help of an example.

What is Message Passing?

Message passing is a fundamental concept in Object-Oriented Programming (OOP) where objects communicate with each other by sending and receiving information called messages.

In OOP, an object sends a message to another object to invoke one of its methods (functions). This allows objects to interact, request services, or exchange data without exposing their internal details.

Explanation with Example

Consider two objects: Car and Driver. The Driver wants the Car to start.

- The Driver object sends a message to the Car object to invoke its start() method.
- The Car receives the message and executes the start() method.

Simple C++ Example

cpp
Copy code

```
#include <iostream>
using namespace std;

class Car {
public:
    void start() {
        cout << "Car has started." << endl;
    }
};

class Driver {
public:
    void drive(Car &car) {
        // Driver sends a message to car to start
        car.start();  // message passing: invoking car's start method
    }
};

int main() {
    Car myCar;
    Driver driver;

    driver.drive(myCar);  // Driver sends message to Car to start it

    return 0;
}
```

Q20. What is ‘this’ pointer ? Explain the significance of ‘this’ pointer with the help of an example program.

What is the this Pointer?

In C++, the this pointer is an implicit pointer available inside all non-static member functions of a class. It points to the object that invoked the member function.

Simply put, this points to the current object for which the member function is called.

Significance of this Pointer

- It helps differentiate between member variables and parameters or local variables when they have the same name.
- It allows member functions to return the calling object itself.
- It is useful in chaining function calls.
- It can be used to pass the current object as a parameter to other functions.

Example Program Using this Pointer

cpp
Copy code

```
#include <iostream>
using namespace std;

class Box {
private:
    int length, width;

public:
    // Constructor with parameters named same as member variables
    Box(int length, int width) {
        // Using this pointer to distinguish member variables from parameters
        this->length = length;
        this->width = width;
    }

    void display() {
        cout << "Length: " << length << ", Width: " << width << endl;
    }

    // Function returning the current object using this pointer
    Box& setLength(int length) {
        this->length = length;
        return *this;  // returning the current object by dereferencing this pointer
    }

    Box& setWidth(int width) {
        this->width = width;
        return *this;
    }
};

int main() {
    Box b1(10, 5);
```

```
    b1.display();

    // Using function chaining to set new values
    b1.setLength(15).setWidth(7);

    b1.display();

    return 0;
}
```

Q21).What is the purpose of exception handling ? Explain how exception is handled in C++, with the help of an example.

What is the Purpose of Exception Handling?

Exception handling is a mechanism to handle runtime errors or unexpected events that occur during program execution. Instead of letting the program crash or produce incorrect results, exception handling allows the program to:

- Detect errors gracefully,
- Transfer control to specialized code to handle the error,
- Maintain normal program flow or terminate cleanly.

Why Exception Handling is Important?

- It improves the robustness and reliability of programs.
- Separates error handling code from regular code, making the program cleaner.
- Allows handling of different types of errors using specific handlers.

How Exception Handling is Done in C++?

C++ uses three keywords for exception handling:

- try — Block of code where exceptions may occur.
- throw — Used to raise an exception when an error condition is detected.
- catch — Block to handle the exception thrown.

Basic Syntax:

cpp
Copy code

```
try {
    // Code that might throw an exception
}
catch (ExceptionType e) {
    // Code to handle the exception
}
```

Example Program of Exception Handling

cpp
Copy code

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two integers (dividend and divisor): ";
    cin >> a >> b;

    try {
        if (b == 0) {
            // Throw an exception if divisor is zero
            throw "Division by zero error!";
        }
        int result = a / b;
        cout << "Result: " << result << endl;
    }
    catch (const char* msg) {
        // Handle the exception here
        cout << "Exception caught: " << msg << endl;
    }

    cout << "Program continues after exception handling." << endl;
    return 0;
}
```

Q22). Explain the difference between method overloading and method overriding with example.

Difference Between Method Overloading and Method Overriding

Both method overloading and method overriding are ways to achieve polymorphism in C++, but they work differently.

Feature	Method Overloading	Method Overriding
Definition	Same method name with different parameters in the same class	Redefining a base class method in a derived class with the same signature
Purpose	To provide multiple ways to perform similar tasks with different inputs	To provide specific implementation of a base class method in derived class
Compile time or Run time	Compile-time polymorphism (early binding)	Run-time polymorphism (late binding) (usually with virtual)
Parameters	Must differ in number or type	Must have exactly the same parameter list as base class method
Return type	Can differ	Should be the same or covariant with base class method
Inheritance needed?	No, happens within the same class	Yes, involves base and derived classes

Example of Method Overloading

cpp
Copy code

```
#include <iostream>
using namespace std;
```

```
class Print {
public:
    void display(int i) {
        cout << "Displaying int: " << i << endl;
    }

    void display(double f) {
        cout << "Displaying float: " << f << endl;
    }

    void display(string s) {
        cout << "Displaying string: " << s << endl;
    }
};

int main() {
    Print obj;
    obj.display(5);          // Calls display(int)
    obj.display(3.14);       // Calls display(double)
    obj.display("Hello");    // Calls display(string)
    return 0;
}
```

Example of Method Overriding

cpp
Copy code

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() {   // virtual for run-time polymorphism
        cout << "Base class show() function" << endl;
    }
};

class Derived : public Base {
public:
    void show() override {  // overriding base class method
        cout << "Derived class show() function" << endl;
    }
};

int main() {
    Base* basePtr;
    Derived d;
    basePtr = &d;

    basePtr->show();  // Calls Derived's show() due to overriding and virtual function

    return 0;
}
```

Q23). What is the difference between function overloading and function overriding in C++ ? Explain the usage of these concepts, with suitable example in C++.

Difference Between Function Overloading and Function Overriding in C++

Feature	Function Overloading	Function Overriding
Definition	Multiple functions with the same name but different parameters in the same class	Redefining a base class function with the same signature in a derived class
Purpose	To perform different tasks with the same function name but different inputs	To modify or extend the behavior of a base class function in the derived class
When resolved	Compile time (early binding)	Run time (late binding), if function is virtual
Parameters	Must differ in number or type	Must have exactly the same parameter list
Return type	Can differ	Must be the same or covariant (compatible)
Inheritance required?	No	Yes
Example of use case	Functions like print(int), print(double) in the same class	draw() function overridden in different derived classes

---

Usage with Examples

1. Function Overloading Example

cpp
Copy code

```
#include <iostream>
using namespace std;

class Print {
public:
    void display(int i) {
        cout << "Displaying int: " << i << endl;
    }
    void display(double f) {
        cout << "Displaying float: " << f << endl;
    }
    void display(string s) {
        cout << "Displaying string: " << s << endl;
    }
};

int main() {
    Print p;
    p.display(10);       // Calls display(int)
    p.display(3.14);     // Calls display(double)
    p.display("Hello");  // Calls display(string)
    return 0;
}
```

---

2. Function Overriding Example

cpp
Copy code

```cpp
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() {     // virtual function
        cout << "Base class show function" << endl;
    }
};

class Derived : public Base {
public:
    void show() override {   // overrides Base class method
        cout << "Derived class show function" << endl;
    }
};

int main() {
    Base* basePtr;
    Derived d;
    basePtr = &d;

    basePtr->show();  // Calls Derived's show() due to overriding and virtual function

    return 0;
}
```

Q24). What is operator overloading ? Write a C++ program to overload '+' operator to find the sum of two complex numbers. Support your program with suitable comments.

What is Operator Overloading?

Operator overloading in C++ allows you to redefine the way operators (like +, -, *, etc.) work for user-defined types (like classes). It enables operators to be used with objects, making code more intuitive and readable.

For example, you can overload the + operator to add two complex numbers directly using c1 + c2 instead of calling a separate function.

C++ Program to Overload + Operator for Complex Numbers

cpp
Copy code

```cpp
#include <iostream>
using namespace std;

class Complex {
private:
    float real;
    float imag;

public:
    // Constructor to initialize complex number
    Complex(float r = 0, float i = 0) {
        real = r;
        imag = i;
```

```
    }

    // Overload + operator to add two Complex objects
    Complex operator + (const Complex& obj) {
        Complex temp;
        temp.real = real + obj.real;   // Add real parts
        temp.imag = imag + obj.imag;   // Add imaginary parts
        return temp;
    }

    // Function to display the complex number
    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(3.5, 2.5);
    Complex c2(1.5, 4.5);

    Complex c3 = c1 + c2;  // Using overloaded + operator

    cout << "Sum of complex numbers: ";
    c3.display();

    return 0;
}
```

.

s